Universidad Nacional Autónoma de México

Facultad de Ingeniería

Integrantes:

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Sistemas Distribuidos

Grupo: 01 - Semestre: 2026-2

Proyecto 1:

Caso de estudio Interfaz de paso de mensajes y Cluster HPC

Profesora:

M.I. Elba Karen Saenz Garcia

Fecha de Entrega:

16 de Marzo del 2026

Índice

1 Objetivo: **1**

2 Desarrollo: **1**

2.1 Documentacion del cluster: 2

2.1.1 Actividad 1: 2

2.2 Actividad 2: 20

2.2.1 Programa a) Paso de matrices entre procesos: 20

2.2.2 Programa b) Determinacion de renglos de matriz que tiene la mayor suma de sus elementos: 26

3 Tabla de participacion en el proyecto: **36**

4 Conclusiones: **37**

Referencias **38**

1. Objetivo:

Construir un sistema distribuido de cómputo de alto rendimiento (HPC) y ejecutar programas paralelos que distribuyan el trabajo entre los nodos que lo conforman.

2. Desarrollo:

Para realizacion del proyecto se tiene como funciones disponibles de MPI las siguientes:

- **MPI_Init:** Inicializa el entorno de ejecución de MPI.
- **MPI_Comm_size:** Determina el número total de procesos en el comunicador.
- **MPI_Comm_rank:** Determina el identificador único (rango) del proceso dentro del comunicador.
- **MPI_Send:** Envía un mensaje a otro proceso.
- **MPI_Recv:** Recibe un mensaje de otro proceso.
- **MPI_Finalize:** Finaliza el entorno de ejecución de MPI.
- **MPI_Gather:** Recopila datos de todos los procesos en el comunicador.
- **MPI_Scatter:** Distribuye datos desde un proceso raíz a todos los procesos en el comunicador.
- **MPI_Bcast:** Difunde un mensaje desde un proceso raíz a todos los procesos en el comunicador.
- **MPI_Reduce:** Realiza una operación de reducción (como suma, máximo, etc.) en datos distribuidos entre los procesos y devuelve el resultado al proceso raíz.

De las cuales, para el desarrollo de los programas, solo se usaron las funciones **MPI_Init**, **MPI_Comm_size**, **MPI_Comm_rank**, **MPI_Send**, **MPI_Recv**, **MPI_Scatter**, **MPI_Gather** y **MPI_Finalize**.

2.1. Documentacion del cluster:

2.1.1. Actividad 1:

Para la realizacion del cluster, se siguieron los siguientes pasos:

Aplicaciones y archivos requeridos para la instalación

1-. Instalación de Oracle VirtualBox Manager Este se instala porque es donde se van a crear las máquinas virtuales que conformarán el cluster HPC.

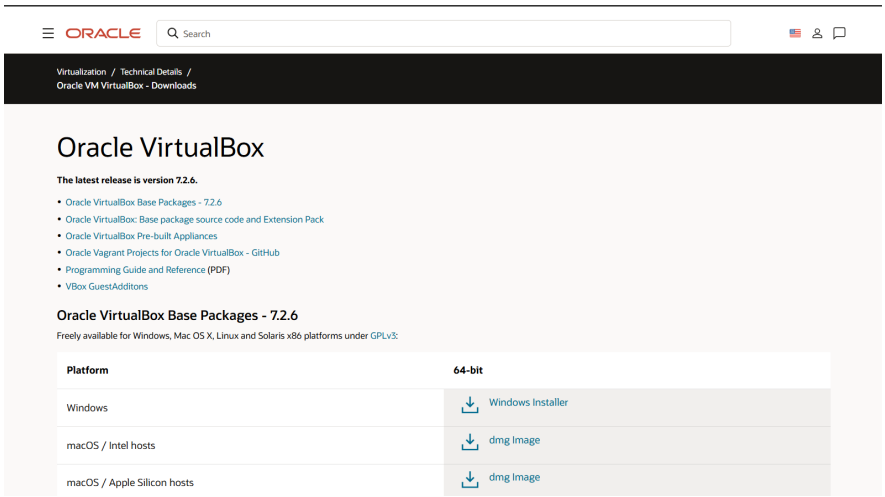

Figura 1: Página de instalación de Oracle VirtualBox Manager

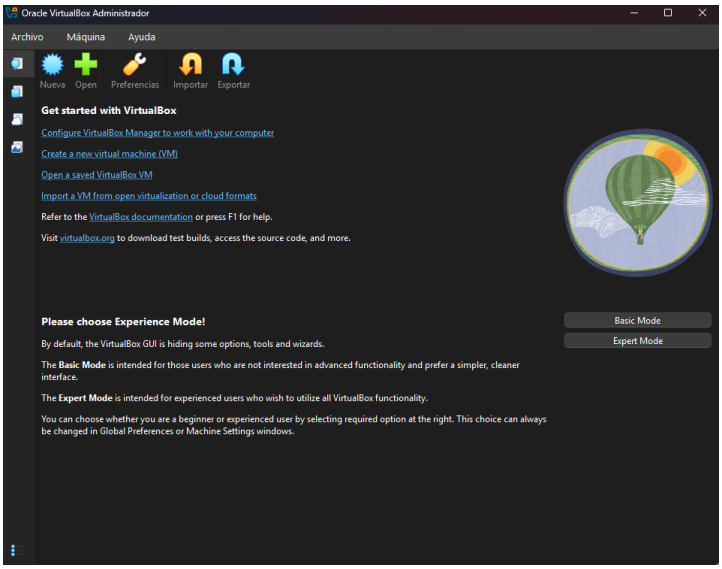

Figura 2: Oracle VirtualBox Manager

2-. Instalación de pelicanhpc version 6.1, que esta mediante un archivo tipo ISO, este se instala porque es la distribución de Linux que se va a usar para crear el cluster HPC.

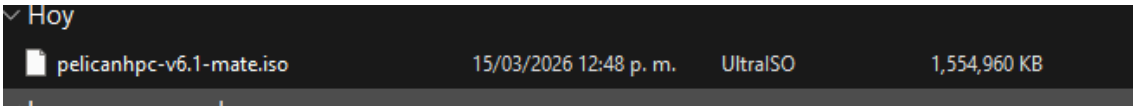

Figura 3: Archivo ISO de PelicanHPC

Creación del servidor

Para la creación del servidor, se crea una máquina virtual en Oracle VirtualBox Manager, asignándole un nombre (en este caso ServidorHPC). Se configuran sus características básicas:

- Tipo de sistema operativo: Linux
- Memoria Base: 2048 MB
- Procesadores: 2

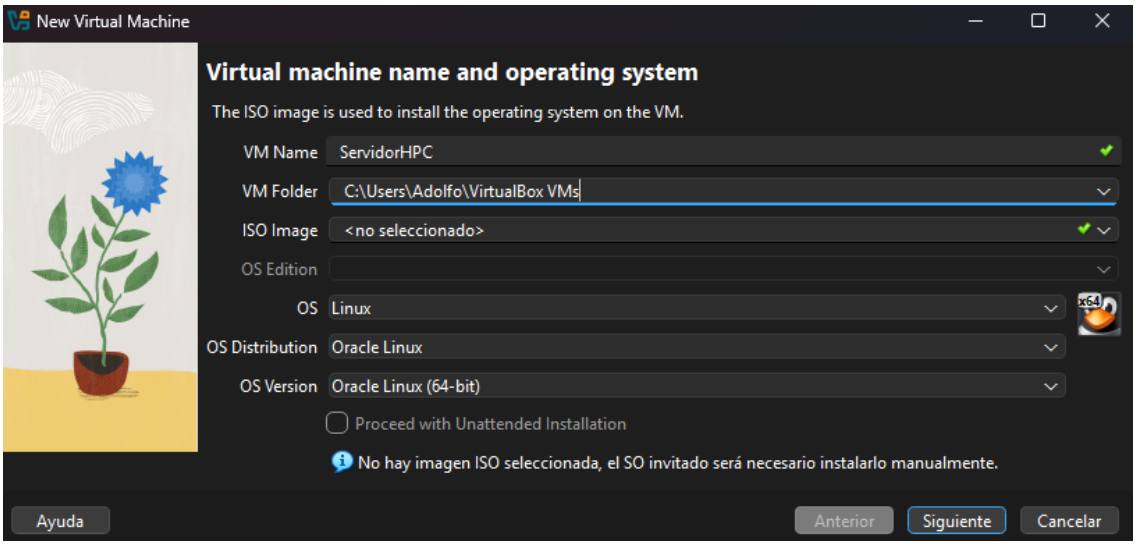

Figura 4: Creación del servidor en Oracle VirtualBox Manager

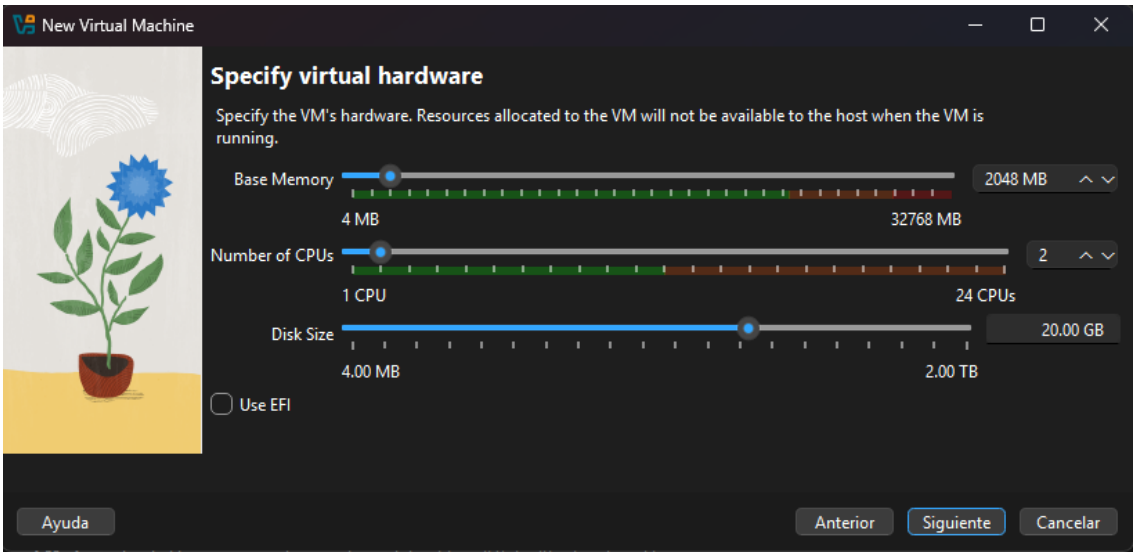

Figura 5: Especificación de características del servidor en Oracle VirtualBox Manager

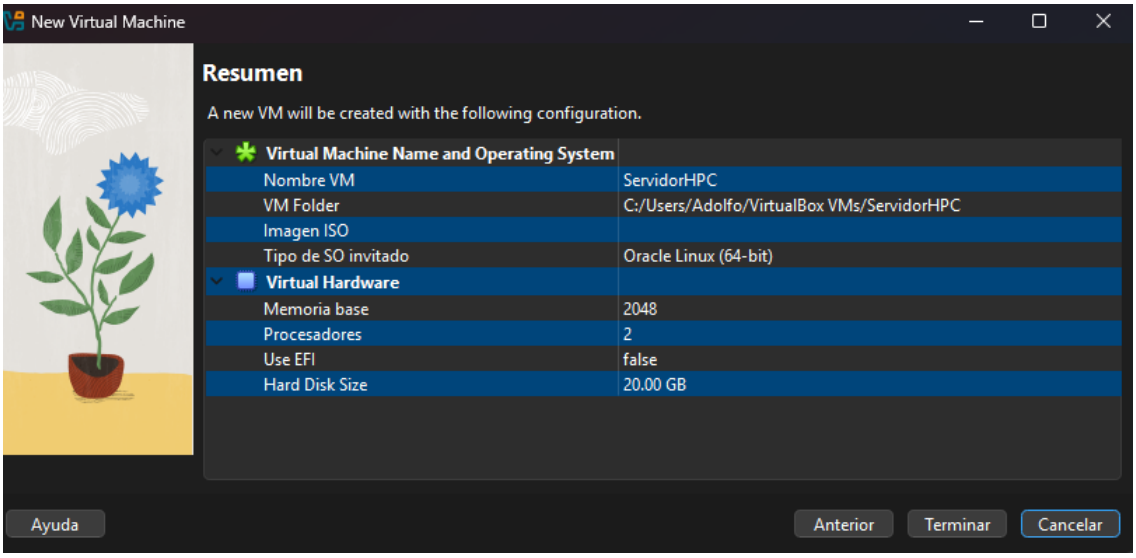

Figura 6: Resumen de características del servidor en Oracle VirtualBox Manager

Como no necesitamos un disco duro virtual lo vamos a eliminarlo al remover el archivo ServidorHPC.vdi que se creó por default.

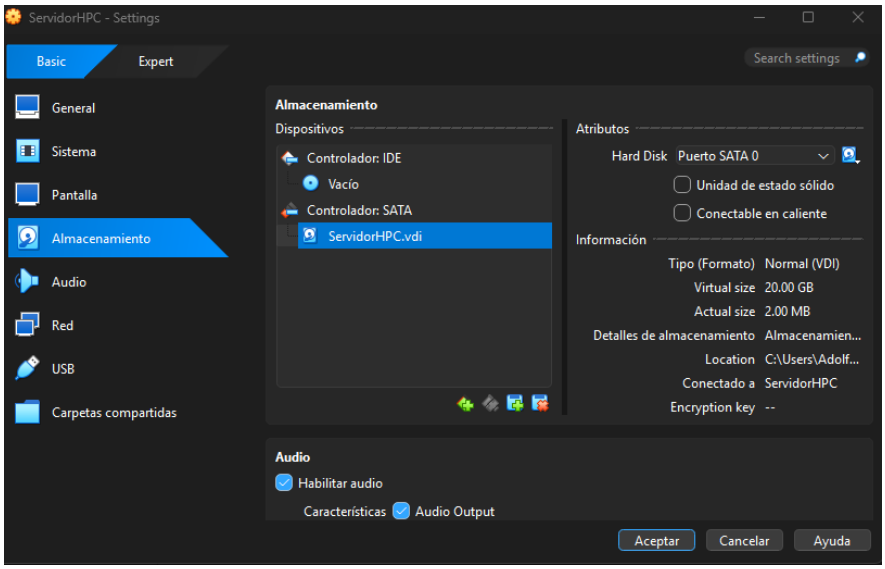

Figura 7: Ubicación del disco duro virtual creado por default en Oracle VirtualBox Manager

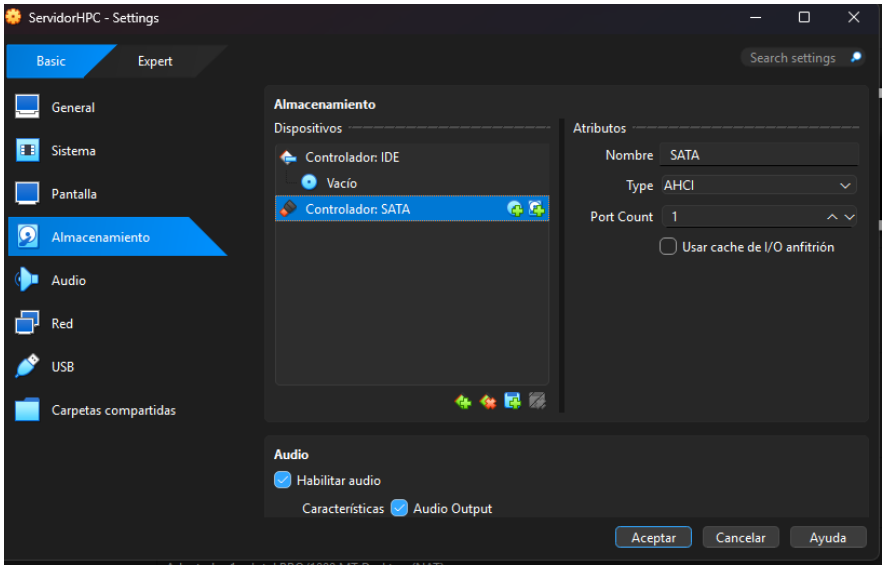

Figura 8: Eliminación del disco duro virtual creado por default en Oracle VirtualBox Manager

Vamos a proseguir con las configuraciónes restantes que nos faltan en nuestro servidor.

Lo primero que vamos a hacer es cambiar la prioridad de arranque de nuestro servidor, vamos a hacer que arranque en .ºptical Drive"para que arranque con el archivo ISO de PelicanHPC que se va a montar posteriormente.

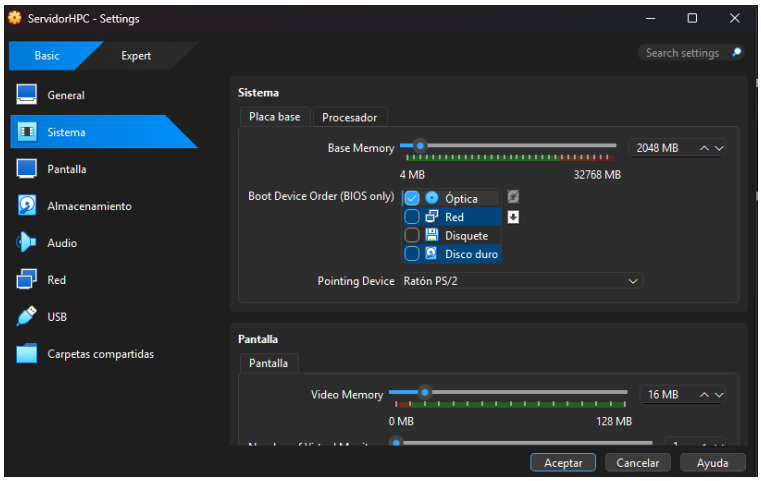

Figura 9: Configuración de la prioridad de arranque en Oracle VirtualBox Manager

Vamos a proseguir a poner el archivo ISO de PelicanHPC en la unidad de CD/DVD de nuestro servidor, para esto vamos a ir a la configuración del servidor, luego a la sección de "Storage", seleccionamos el controlador IDE y luego seleccionamos la unidad de CD/DVD, después seleccionamos la opción de Çhoose a disk filez seleccionamos el archivo ISO de PelicanHPC que se encuentra en nuestra computadora.

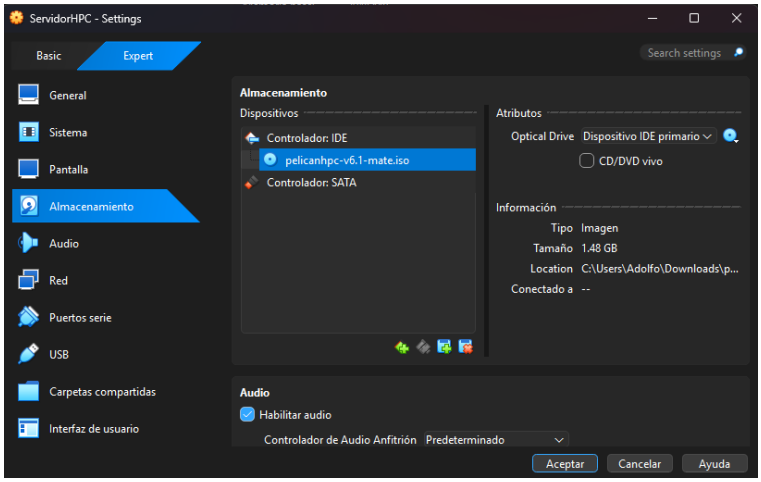

Figura 10: Ponemos el archivo ISO de PelicanHPC en Oracle VirtualBox Manager

Proseguimos con la configuración de la red del servidor, para esto vamos a ir a la configuración del servidor, luego a la sección de "Network", seleccionamos el adaptor 1 y lo dejamos de tipo "NAT", para que podamos acceder a internet desde el servidor.

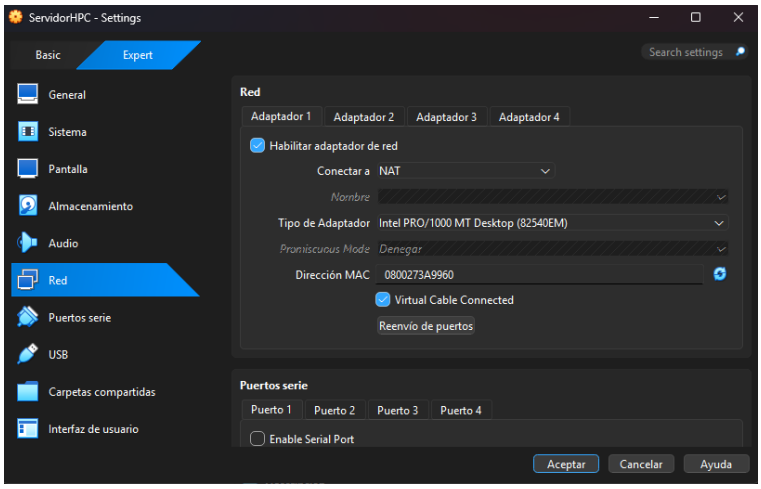

Figura 11: Configuración del adaptador de red 1 en Oracle VirtualBox Manager

Vamos a activar un segundo adaptador de red y lo vamos a dejar en Red interna solo, para que los nodos puedan comunicarse entre sí y con el servidor.

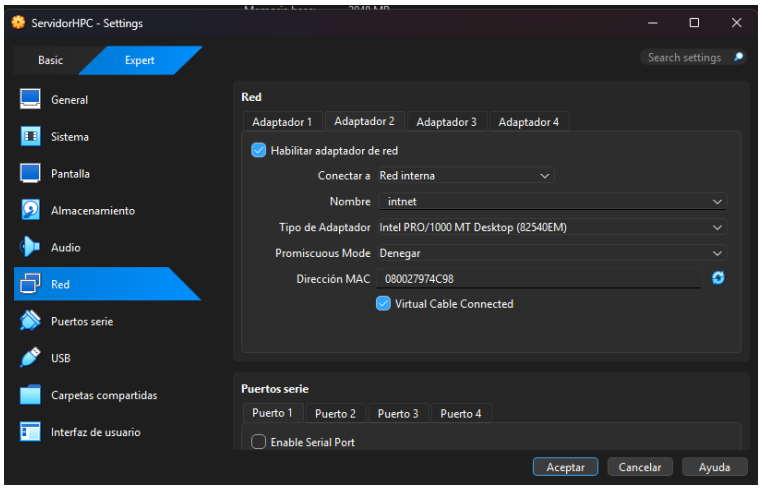

Figura 12: Configuración del adaptador de red 2 en Oracle VirtualBox Manager

Iniciamos nuestra máquina virtual del servidor, siguiendo las propias instrucciones de PelicanHPC Con la única diferencia de que vamos primero a levantar el ambiente gráfico con "startxz luego vamos a crear el cluster con "pelicansetup"

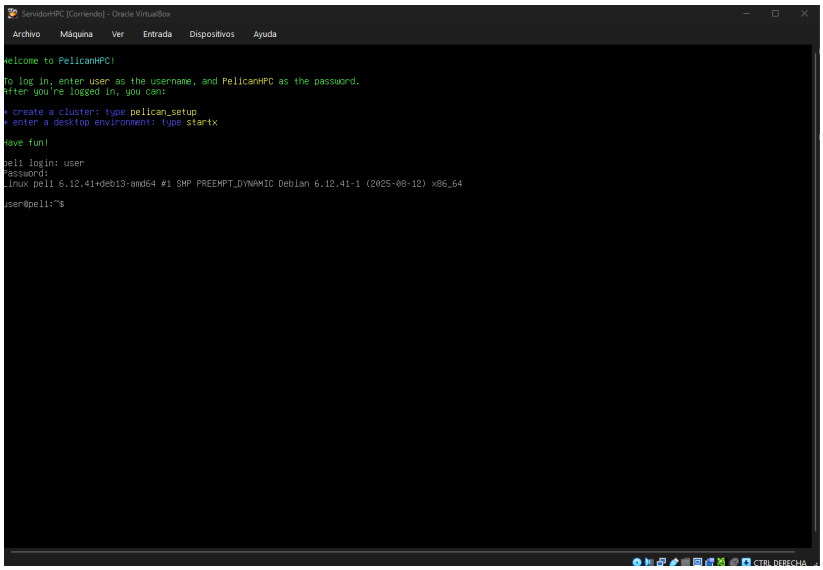

Figura 13: Levantando el ambiente gráfico en el servidor con PelicanHPC

Una vez ya levantando el ambiente gráfico, vamos a crear el cluster con el comando "pelicansetup", este comando se encarga de configurar el cluster HPC, asignando las IPs a los nodos y configurando el servicio de SSH para que los nodos puedan comunicarse entre sí y con el servidor. Donde nos aparecen las dos interfaces de red que tenemos

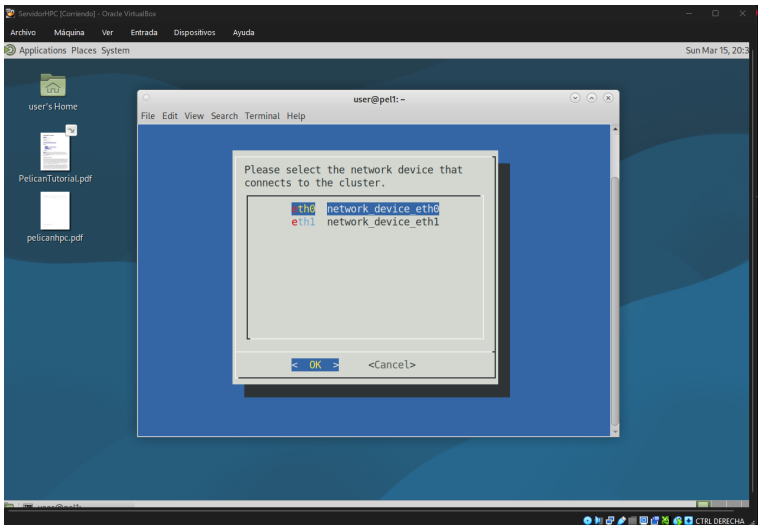

Figura 14: Creando el cluster con PelicanHPC

Elegimos la segunda interfaz de red, que es donde estan los nodos del cluster, y empezamos a configurar nuestro cluster, para finalmente empezar a cargar nuestros nodos.

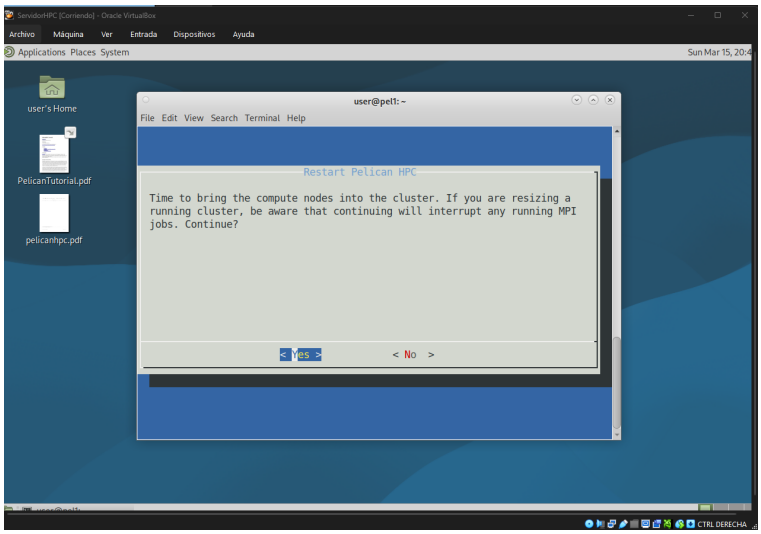

Figura 15: Configurando el cluster con PelicanHPC

Creación de los clientes

Para la creación de los clientes, se crearan dos clientes en Oracle VirtualBox Manager, asignándoles un nombre (en este caso ClienteHPC1 y ClienteHPC2).

Empezando con el ClienteHPC1, se configuran sus características básicas:

- Tipo de sistema operativo: Windows
- Memoria Base: 2048 MB
- Procesadores: 1

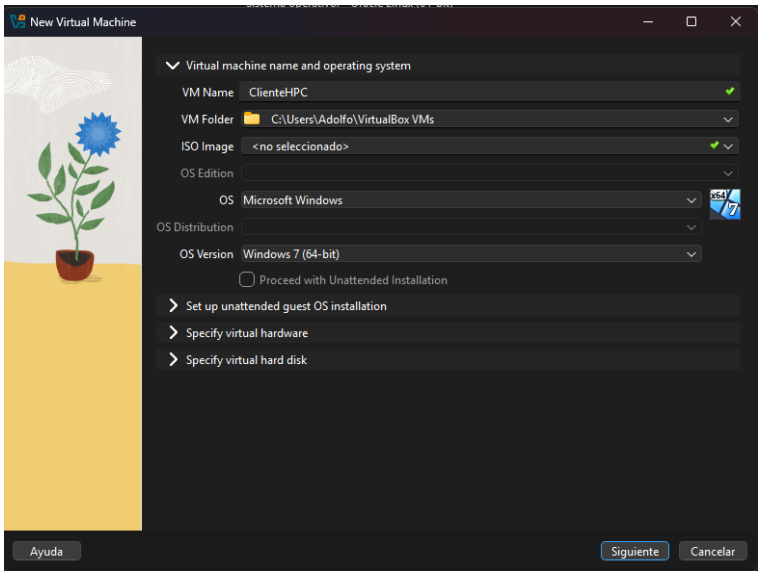

Figura 16: Sistema Operativo del ClienteHPC1 en Oracle VirtualBox Manager

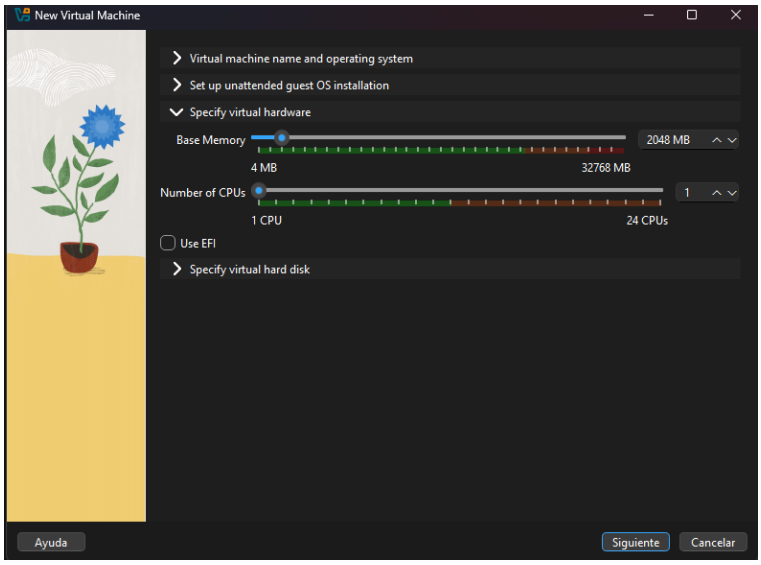

Figura 17: Memoria Base y número de procesadores del ClienteHPC1 en Oracle VirtualBox Manager

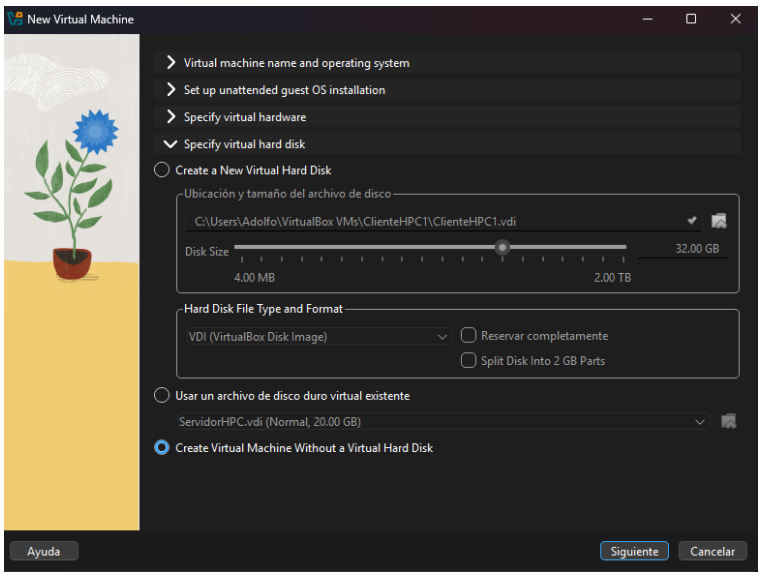

Figura 18: Se crea sin disco duro virtual el ClienteHPC1 en Oracle VirtualBox Manager

Vamos ahora a configurar la prioridad de arranque del ClienteHPC1, para que arranque en "Networkz pueda arrancar desde el servidor del cluster.

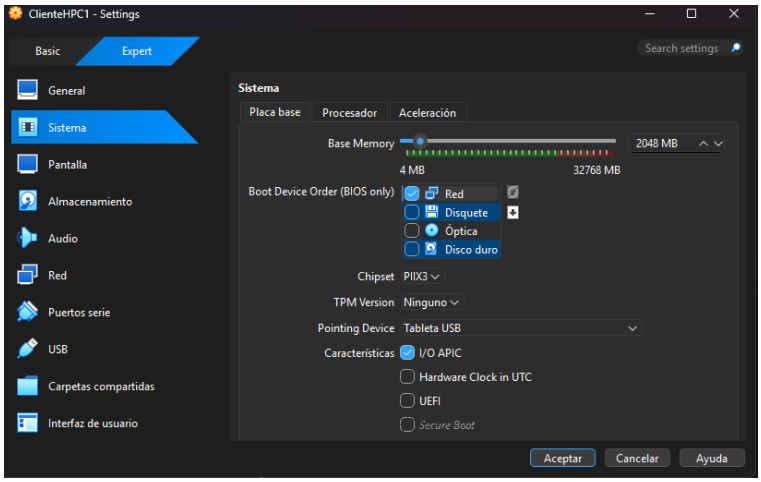

Figura 19: Configuración de la prioridad de arranque del ClienteHPC1 en Oracle VirtualBox Manager

Proseguimos con la configuración de la red del ClienteHPC1, para esto vamos a ir a la configuración del cliente, luego a la sección de "Network", seleccionamos el adaptador 1 y lo dejamos en Red interna solo, para que el cliente pueda comunicarse con el servidor y los demás nodos del cluster.

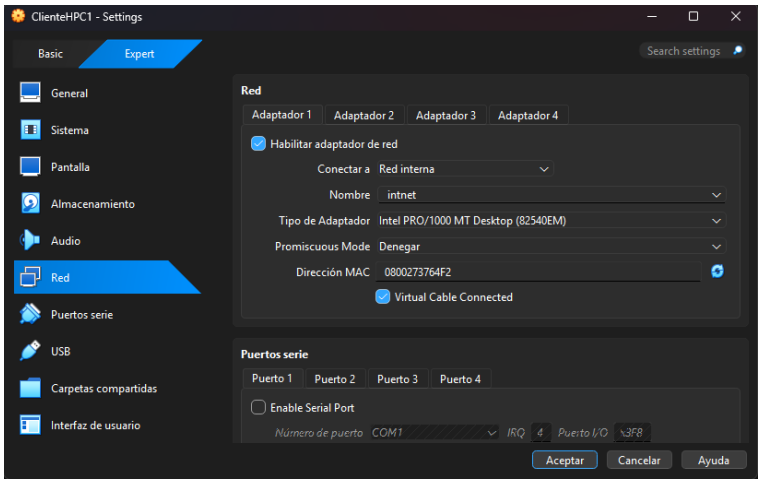

Figura 20: Configuración del adaptador de red del ClienteHPC1 en Oracle VirtualBox Manager

Inicializamos la máquina virtual, pero como inicializa por red, tenemos que esperar unos momentos hasta que se conecte el servidor y queda de la siguiente manera:

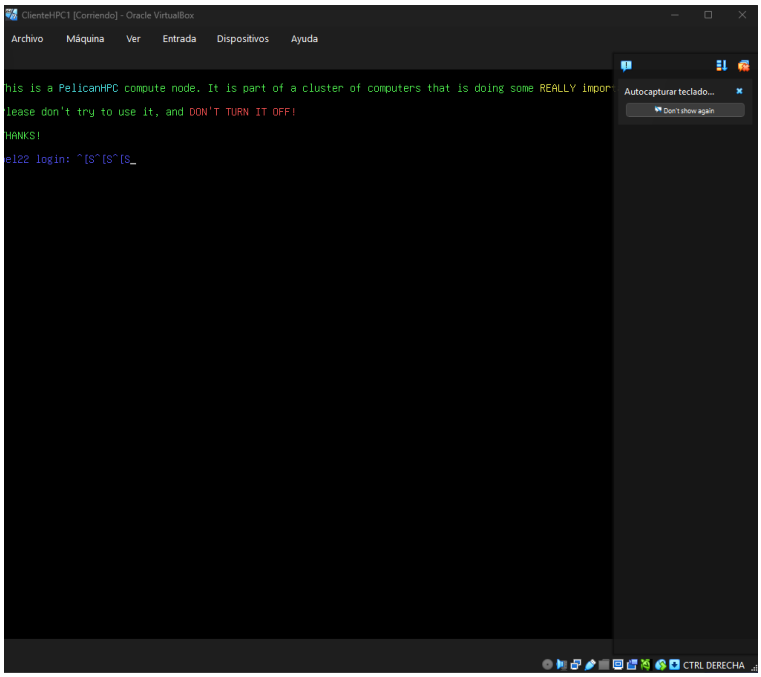

Figura 21: Inicialización del ClienteHPC1 en Oracle VirtualBox Manager

Finalmente el servidor detecte que este cliente o nodo ha sido creado, por eso marca que al momento solo nay 1 nodo disponible

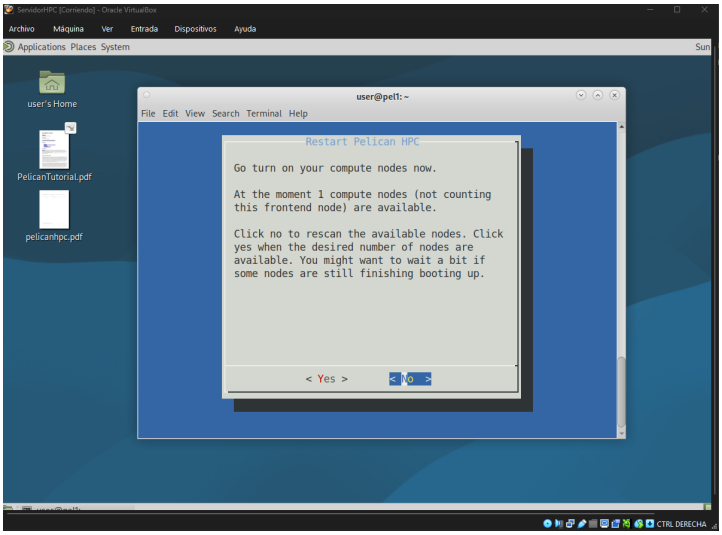

Figura 22: Servidor detecta el ClienteHPC1 en Oracle VirtualBox Manager

Vamos a proceder a crear el ClienteHPC2, el cual se va a configurar de la misma manera que el ClienteHPC1.

Por lo que se mostrara directamente como ya se ve la maquina virtual del ClienteHPC2 y como se ve reflejado en el servidor

Figura 23: ClienteHPC2 en Oracle VirtualBox Manager

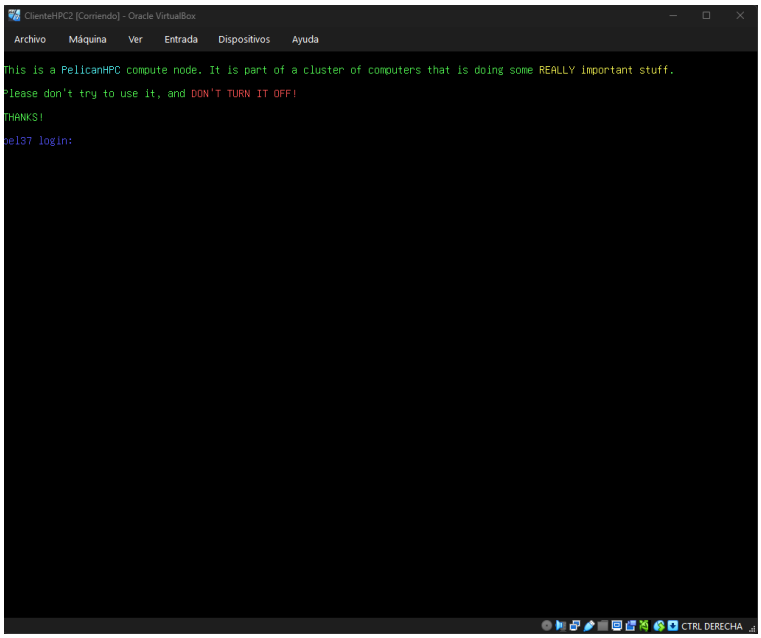

Figura 24: Inicialización del ClienteHPC2 en Oracle VirtualBox Manager

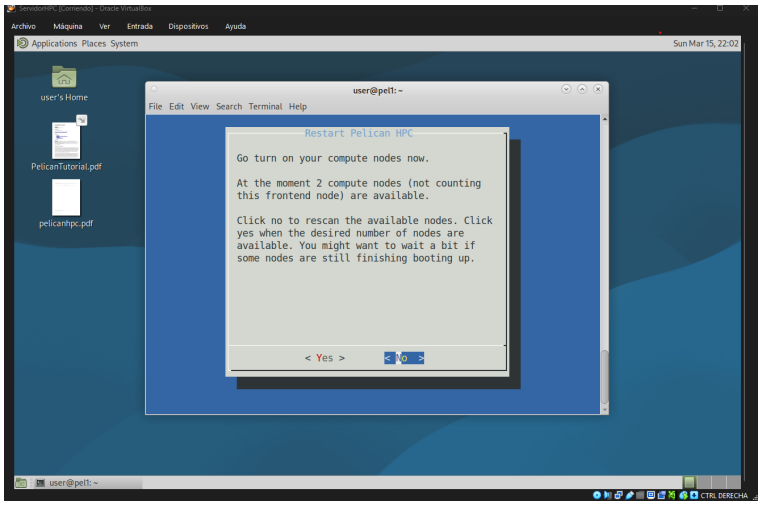

Figura 25: Servidor detecta el ClienteHPC2 en Oracle VirtualBox Manager

Creación del programa

Se nos pidio hacer lo siguiente: La ejecución de un programa paralelo que u lice al menos 6 procesos. Para ello, desarrollar un programa en el que el proceso 0 envíe un dato diferente a otros cinco procesos. Cada proceso receptor deberá imprimir: El nombre de la máquina o nodo, su iden ficador de proceso y el dato recibido.

Para esto cree 5 máquinas virtuales más, con las mismas características que los clientes anteriores, pero con la prioridad de arranque en .ºptical Drive"para que arranquen con el archivo ISO de PelicanHPC que se encuentra en el servidor, y con la red en Red interna solo para que puedan comunicarse con el servidor y los demás nodos del cluster.

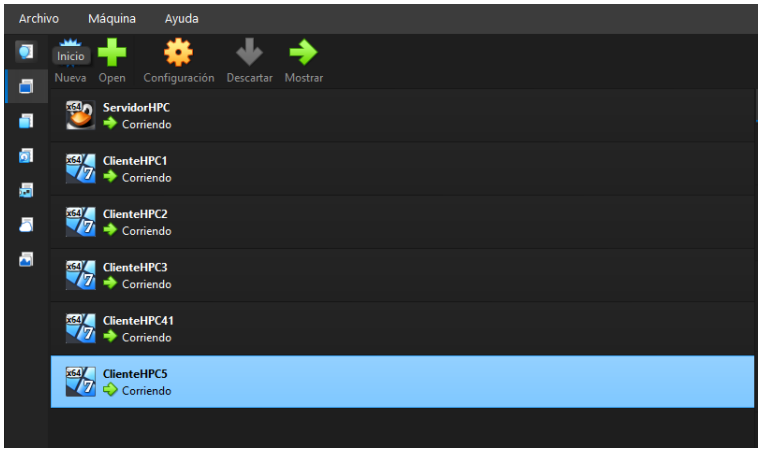

Figura 26: Clientes creados en Oracle VirtualBox Manager

Observamos que el servidor detecta que se han creado los nuevos nodos, por lo que ahora tenemos un total de 6 nodos disponibles para ejecutar nuestro programa paralelo.

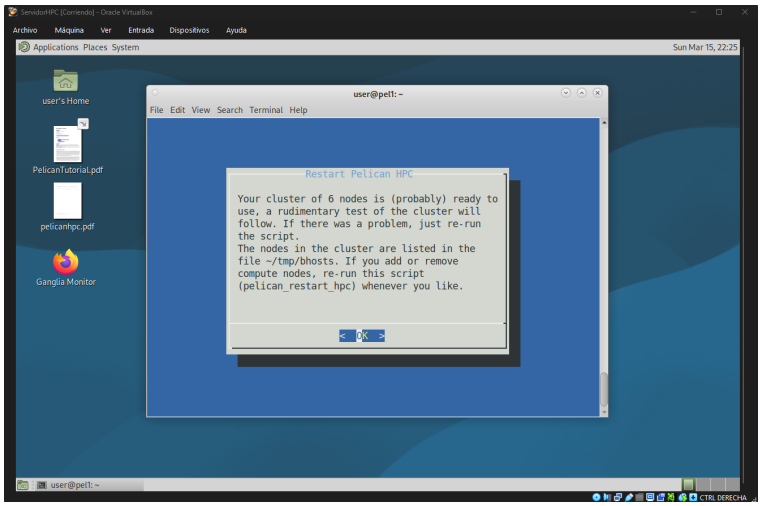

Figura 27: Servidor detecta el ClienteHPC3, ClienteHPC4, ClienteHPC5 y ClienteHPC6 en Oracle VirtualBox Manager

El código es el siguiente:

Listing 1: Distribucion de datos desde el proceso maestro a los nodos clientes con MPI

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char** argv) {
    int rank, size;
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;

    // Inicializar MPI
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(processor_name, &name_len);

    // Verificacion de seguridad
    if (size < 6) {
        if (rank == 0) {
```

```c
            printf("Error: Se necesitan 6 procesos para los 6 nodos.\n");
        }
        MPI_Finalize();
        return 0;
    }

    if (rank == 0) {
        // Logica del Servidor (Proceso 0)
        printf("Soy el maestro (Proceso 0) en el nodo: %s\n", processor_name);
        printf("Enviando un dato unico a los 5 nodos clientes...\n\n");

        // Enviar a los procesos del 1 al 5
        for (int dest = 1; dest <= 5; dest++) {
            int dato_a_enviar = dest * 100; // Datos: 100, 200, 300, 400, 500
            MPI_Send(&dato_a_enviar, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
        }
    } else if (rank >= 1 && rank <= 5) {
        // Logica de los Clientes (Procesos 1 al 5)
        int dato_recibido;

        MPI_Recv(&dato_recibido, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);

        printf("Nodo cliente: %s | ID de proceso: %d | Dato recibido: %d\n",
               processor_name, rank, dato_recibido);
    }

    // Finalizar MPI
    MPI_Finalize();
    return 0;
}
```

Este programa escrito en C utiliza la biblioteca MPI (*Message Passing Interface*) para comunicar diferentes procesos que pueden estar ejecutándose en distintas computadoras o núcleos (nodos). El código sigue un modelo clásico de "maestro-cliente". A continuación, se detalla su funcionamiento paso a paso:

- **Preparación inicial:** Las funciones `MPI_Init`, `MPI_Comm_rank` y `MPI_Comm_size` se encargan de encender el sistema de comunicación. A cada proceso se le asigna un número identificador único llamado `rank` (el proceso 0 será nuestro maestro, y los demás serán los clientes). También se cuenta cuántos procesos hay en total (`size`) y se guarda el nombre del equipo donde se está ejecutando mediante `MPI_Get_processor_name`.

- **Verificación de seguridad:** El programa comprueba mediante un `if (size < 6)` que se hayan lanzado al menos 6 procesos en total (1 maestro y 5 clientes). Si el usuario ejecutó el programa con menos procesos, el maestro muestra un mensaje de error y el programa se detiene de forma segura.

- **La labor del Maestro (Proceso 0):** Si el `rank` es 0, este proceso toma el rol de jefe. Su trabajo es usar un bucle `for` para enviar un número matemático simple y único a cada cliente. Usando la instrucción `MPI_Send`, le envía un 100 al proceso 1, un 200 al proceso 2, un 300 al proceso 3, y así sucesivamente.

- **La labor de los Clientes (Procesos del 1 al 5):** Si el `rank` está entre el 1 y el 5, el proceso actúa como cliente. Su única tarea es esperar a que el maestro le envíe la información utilizando la instrucción `MPI_Recv`. Una vez que el dato llega, el cliente imprime en la pantalla dónde se está ejecutando, cuál es su número de proceso y qué número recibió exactamente.

- **Cierre del programa:** Al final, la instrucción `MPI_Finalize()` se asegura de cerrar las conexiones de red y apagar el entorno MPI de manera limpia para no dejar procesos atascados en la memoria.

El resultado del código es el siguiente, primero me gustaria mencionar los procesos que tenemos con el comando çat /etc/hosts"para mostrar los nodos que tenemos disponibles, que son los siguientes:

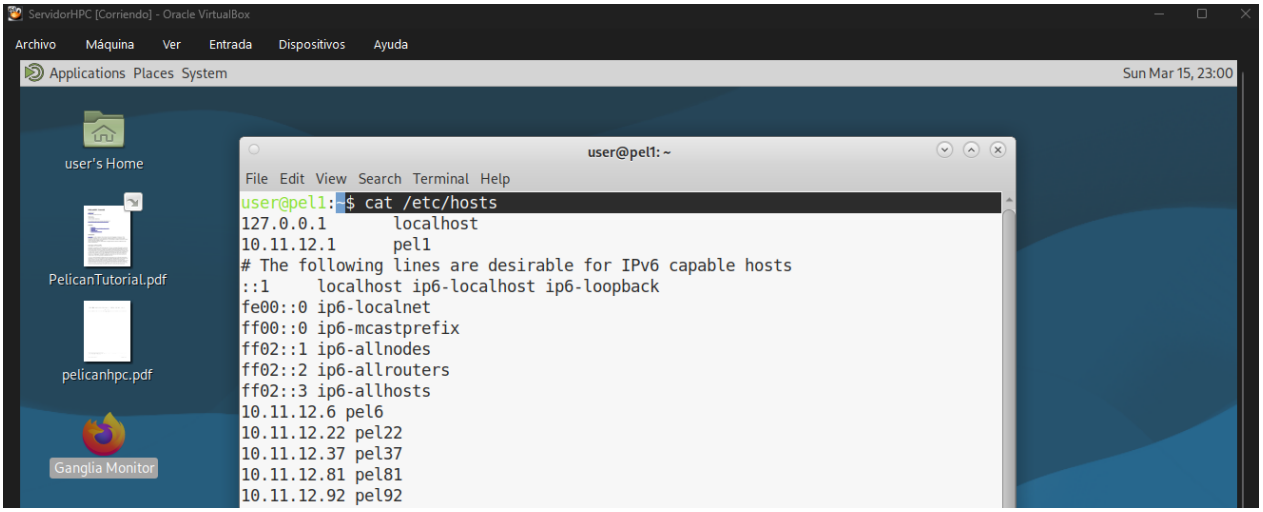

Figura 28: Nodos disponibles en el cluster con el comando çat /etc/hosts"

Este es lo que imprime el programa anteriormente descrito:

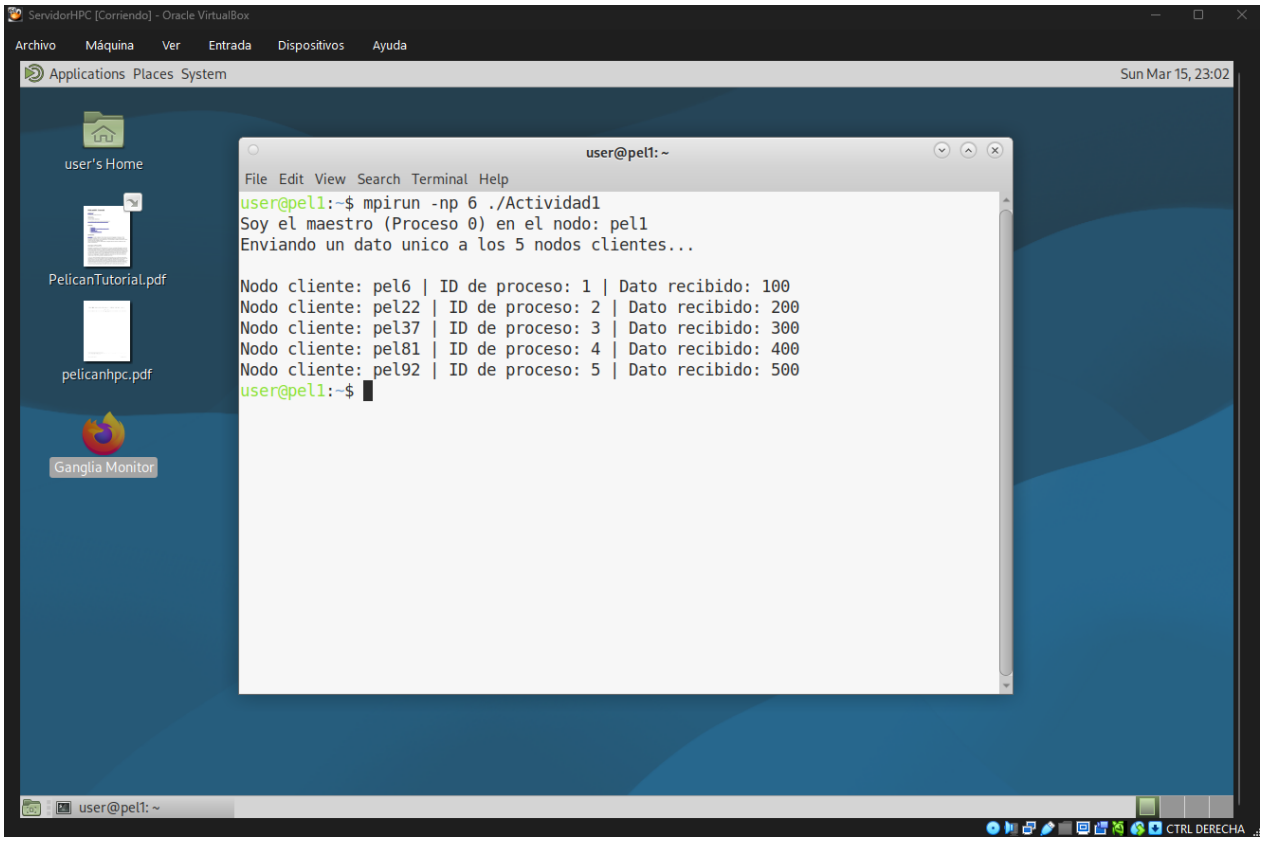

Figura 29: Resultado del programa paralelo en el cluster

Como se puede observar usamos los mismos nodos, al ver el nombre de los nodos, ya que son los mismos que podemos ver en la imagen anterior, corroborando el correcto funcionamiento del programa

2.2. Actividad 2:

2.2.1. Programa a) Paso de matrices entre procesos:

Enunciado: Implementar un programa que realice el paso de dos matrices de enteros (A y B) entre varios procesos, utilizando np procesos, tal como se indica en la figura proporcionada.

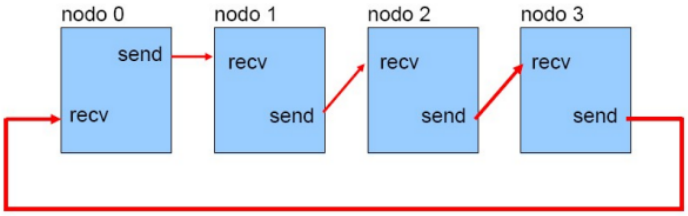

Figura 30: Diagrama de paso de matrices entre procesos

Cada vez que un proceso reciba las matrices A y B, deberá: Sumar ambas matrices, almacenar el resultado en la matriz A y enviar las matrices al siguiente proceso.

El proceso 0 deberá imprimir las matrices A y B con sus valores iniciales, el resultado final obtenido después del paso por todos los procesos.

Resolución : Al inicio del programa tenemos lo siguiente:

Listing 2: Inclusiones, definiciones y funcion de impresion

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define N 4
#define M 4

void print_matrix(char* titulo, int *mat, int rows, int cols) {
    printf("  %s:\n", titulo);
    for (int i = 0; i < rows; i++) {
        printf("  [");
        for (int j = 0; j < cols; j++) {
            printf("%4d", mat[i * cols + j]);
            if (j < cols - 1) printf(", ");
        }
        printf(" ]\n");
    }
}
```

En esta primera parte del código, se incluyen las librerías estándar necesarias para la ejecución, tales como `<mpi.h>` para utilizar el estándar de paso de mensajes, así como `<stdio.h>` y `<stdlib.h>` para la entrada/salida de datos en consola y la asignación dinámica de memoria. Además, se definen las constantes `N` y `M` con un valor de 4, las cuales determinan el tamaño de las matrices enteras que procesará el anillo (4×4).

Inmediatamente después, se declara la función auxiliar `print_matrix()`. Esta función recibe el título de la matriz, el puntero a los datos y sus dimensiones (`rows` y `cols`). Su propósito es iterar sobre los elementos del arreglo mediante un ciclo anidado para mostrarlos en pantalla con un formato de cuadrícula uniforme, corchetes y espaciado, lo que facilita la visualización y comprobación de las operaciones matemáticas en la terminal.

Posterior a esto, se da inicio a la función principal `main()`:

Listing 3: Inicializacion del entorno y asignacion de memoria

```
int main(int argc, char **argv) {
    int np, id;
    char hostname[256];

    // Asignacion de memoria para A y B (matrices completas)
    int *A = (int *)malloc(N * M * sizeof(int));
    int *B = (int *)malloc(N * M * sizeof(int));

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &np);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    gethostname(hostname, sizeof(hostname));

    if (np < 2) {
        if (id == 0) printf("Error: Se necesitan al menos 2 procesos\n");
        MPI_Finalize();
        return 1;
    }
```

Primero se declaran las variables necesarias para el programa, como el número de procesos (`np`), el identificador del proceso (`id`) y un arreglo para almacenar el nombre del host donde se ejecuta cada proceso (`hostname`). Además, se reserva la memoria dinámica para los punteros de las matrices enteras A y B, las cuales tendrán un tamaño total de $N \times M$.

Luego se inicializa el entorno de ejecución de MPI con `MPI_Init()`, se obtiene el número total de procesos con `MPI_Comm_size()` y el identificador único de cada proceso con `MPI_Comm_rank()`. También se obtiene el nombre del nodo físico utilizando la función `gethostname()`. Se implementa una validación para asegurar

que existan al menos 2 procesos, ya que es el mínimo indispensable para conformar la topología de anillo.

Para la impresión de las matrices durante la ejecución, se desarrolló una función auxiliar:

Listing 4: Funcion de impresion de matriz

```c
void print_matrix(char* titulo, int *mat, int rows, int cols) {
    printf("  %s:\n", titulo);
    for (int i = 0; i < rows; i++) {
        printf("  [");
        for (int j = 0; j < cols; j++) {
            printf("%4d", mat[i * cols + j]);
            if (j < cols - 1) printf(", ");
        }
        printf(" ]\n");
    }
}
```

Esta función recibe el título de la matriz, el puntero a los datos y sus dimensiones, iterando mediante un ciclo anidado para mostrar los elementos con un formato de espaciado uniforme.

Posteriormente, comienza la lógica específica del proceso raíz:

Fragmento de codigo de generacion de matriz (C):

Listing 5: Generacion de matrices e inicio del anillo en proceso 0

```c
// --- LOGICA DEL PROCESO 0 (Inicia y Termina el ciclo) ---
if (id == 0) {
    // Inicializar matrices A y B
    for (int i = 0; i < N * M; i++) {
        A[i] = i + 1; // Valores secuenciales
        B[i] = 2;     // Valor constante para que sea facil verificar la
            suma
    }

    printf("==========================================================\n");
    printf("  INICIO DEL PROGRAMA - %d procesos\n", np);
    printf("==========================================================\n");
    printf("  [P0] Nodo: %s | Genera e imprime matrices iniciales\n\n",
        hostname);
    print_matrix("Matriz A (Inicial)", A, N, M);
    printf("\n");
    print_matrix("Matriz B (Constante)", B, N, M);
```

```
        printf("==========================================================\n");

        // Enviar A y B al Proceso 1
        printf("  [P0] Nodo: %s | Enviando matrices a P1...\n", hostname);
        MPI_Send(A, N * M, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Send(B, N * M, MPI_INT, 1, 1, MPI_COMM_WORLD);
```

Cuando la condición evalúa al proceso 0, este se encarga de inicializar la matriz A con valores numéricos secuenciales y la matriz B con un valor constante (2) para facilitar la verificación del cálculo distribuido. Tras imprimir los valores iniciales en consola, el proceso 0 envía ambas matrices completas al Proceso 1 utilizando la función `MPI_Send()`.

El primer `MPI_Send()` envía la matriz A, especificando su puntero, la cantidad total de elementos ($N \times M$), el tipo de dato (`MPI_INT`), el proceso destino (1), una etiqueta de mensaje (0) y el comunicador. Inmediatamente después, envía la matriz B utilizando la etiqueta de mensaje (1).

Al mismo tiempo, el resto de los procesos que conforman el anillo ejecutan la lógica de recepción, suma y reenvío:

Listing 6: Recepcion, suma local y reenvio en procesos trabajadores

```
    // --- LOGICA DE LOS PROCESOS TRABAJADORES (1 a np-1) ---
    else {
        // Recibir del proceso anterior (id - 1)
        MPI_Recv(A, N * M, MPI_INT, id - 1, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
        MPI_Recv(B, N * M, MPI_INT, id - 1, 1, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);

        printf("  [P%d] Nodo: %s | Recibio matrices de P%d. Sumando A = A + B
            ...\n", id, hostname, id - 1);

        // Sumar ambas matrices y almacenar en A
        for (int i = 0; i < N * M; i++) {
            A[i] = A[i] + B[i];
        }

        // Determinar el siguiente nodo (si es el ultimo, envia al 0)
        int next_node = (id + 1) % np;

        // Enviar al siguiente nodo
        MPI_Send(A, N * M, MPI_INT, next_node, 0, MPI_COMM_WORLD);
```

```
        MPI_Send(B, N * M, MPI_INT, next_node, 1, MPI_COMM_WORLD);
        printf("  [P%d] Nodo: %s | Matrices enviadas a P%d.\n", id, hostname,
            next_node);
    }
```

En este bloque, cada proceso intermedio utiliza `MPI_Recv()` para capturar las matrices enviadas por su predecesor (`id - 1`). Una vez recibidos los datos, se ejecuta un ciclo `for` lineal que recorre los arreglos unidimensionales para sumar cada celda de la matriz B a su correspondiente en la matriz A, sobreescribiendo el valor directamente en A.

Para continuar con la topología de anillo, se calcula la identidad del siguiente proceso destino mediante la operación de módulo aritmético `next_node = (id + 1) % np`. Esta fórmula garantiza que el último proceso apunte de vuelta al proceso 0. Finalmente, se despachan los datos al nodo calculado.

Por último, el programa se cierra de regreso en el flujo del proceso 0, el cual se había quedado bloqueado esperando la conclusión del ciclo:

Listing 7: Recepcion de resultados y finalizacion en proceso 0

```
        // Esperar a recibir el resultado del ultimo proceso
        MPI_Recv(A, N * M, MPI_INT, np - 1, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
        MPI_Recv(B, N * M, MPI_INT, np - 1, 1, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);

        // Imprimir resultado final
        printf("\n");
        printf("============================================================\n");
        printf("  RESULTADO FINAL - Ciclo completado\n");
        printf("============================================================\n");
        printf("  [P0] Nodo: %s | Recibio resultado de P%d\n\n", hostname, np
            - 1);
        print_matrix("Matriz A (Final despues de todas las sumas)", A, N, M);
        printf("============================================================\n");

    }

    free(A);
    free(B);
    MPI_Finalize();
    return 0;
}
```

El proceso 0 emplea llamadas a `MPI_Recv()` indicando explícitamente que espera el mensaje proveniente del último nodo del clúster (`np - 1`). Al recibir las matrices modificadas tras pasar por todos los procesos trabajadores, el proceso 0 imprime la matriz A resultante que contiene la sumatoria global acumulada. Para finalizar, se libera la memoria reservada para ambas matrices mediante `free()` y se cierra de forma segura el entorno de paso de mensajes con `MPI_Finalize()`.

2.2.2. Programa b) Determinacion de renglos de matriz que tiene la mayor suma de sus elementos:

Enunciado: Se tiene una matriz A de tamaño $N \times M$ que contiene números enteros. Desarrollar un programa distribuido utilizando MPI que determine qué renglón de la matriz tiene la mayor suma de sus elementos.

El programa deberá distribuir el trabajo entre varios procesos de manera que cada proceso analice una parte de la matriz.

El proceso 0 debe generar o inicializar la matriz A; la matriz deberá distribuirse entre los procesos utilizando comunicaciones colectivas de MPI. Cada proceso calculará la suma de los elementos de los renglones que le correspondan. Cada proceso determinará la fila con mayor suma dentro de su bloque local.

El proceso 0 deberá imprimir: el índice de la fila con mayor suma y el valor de dicha suma.

Suponer que N es divisible entre el número de procesos.

Resolucion : Al iniciar el programa tenemos las siguientes lineas:

Listing 8: Generacion de matriz por el proceso 0 e impresion

```
int main(int argc, char **argv) {
int np, id;
int *A = NULL;
int *local_A;
int rows_per_proc;
char hostname[256];

MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &np);
MPI_Comm_rank(MPI_COMM_WORLD, &id);
gethostname(hostname, sizeof(hostname));

rows_per_proc = N / np;

if (id == 0 && N % np != 0) {
    printf("Error: N (%d) debe ser divisible entre np (%d)\n", N, np);
    MPI_Finalize();
    return 1;
}
```

Primero se declaran las variables necesarias para el programa, como el número de procesos (`np`), el identificador del proceso (`id`), un puntero para la matriz A y un puntero para el bloque local de cada proceso (`local_A`), una variable para almacenar el número de filas que cada proceso analizará (`rows_per_proc`) y un arreglo para almacenar el nombre del host donde se ejecuta cada proceso.

Luego se inicializa el entorno de ejecución de MPI con `MPI_Init()`, se obtiene el número total de procesos con `MPI_Comm_size()` y el identificador único de cada proceso con `MPI_Comm_rank()`. También se obtiene el nombre del host utilizando la función `gethostname()`.

Después de esto, se calcula el número de filas que cada proceso analizará dividiendo el número total de filas N entre el número de procesos `np`. Se verifica que N sea divisible entre `np`, y si no lo es, el proceso 0 imprime un mensaje de error y finaliza el programa.

Para la creacion de este programa, primero definimos el tamaño de la matriz $N \times M$ y el número de procesos a utilizar.

Para esto el tamaño de N, es de 8 y el tamaño de M es de 5, el proceso 0 se encarga de generar la matriz A con números enteros aleatorios.

Fragmento de codigo de generacion de matriz (C):

Listing 9: Generacion de matriz por el proceso 0 e impresion

```c
    if (id == 0) {
        A = (int *)malloc(N * M * sizeof(int));
        srand(42);
        for (int i = 0; i < N * M; i++)
            A[i] = rand() % 100;

        separador();
        printf("  FILA CON MAYOR SUMA - %d procesos, Matriz %dx%d\n", np, N, M
            );
        separador();
        printf("\n  [P0] Nodo: %s | Genera la matriz A\n\n", hostname);
        print_matrix(A, N, M);
        separador();
        printf("  [P0] Nodo: %s | Distribuyendo %d filas a cada proceso (
            MPI_Scatter)\n", hostname, rows_per_proc);
        separador();
    }
```

Para esto, cuando la condicion evalua al proceso 0 se encarga de generar la matriz A con números enteros aleatorios, utilizando la función `rand()` y una semilla fija. Despues de esto, el proceso 0 imprime la matriz generada, llamando a la funcion `print_matrix()`, la cual para imprimir la matriz, hace uso de un ciclo anidado para recorrer los elementos de la matriz y mostrar los elementos de la misma en formato de filas y columnas.

Listing 10: Funcion de impresion de matriz

```c
void print_matrix(int *mat, int rows, int cols) {
    printf("          ");
    for (int j = 0; j < cols; j++)
        printf("Col%-3d", j);
    printf("\n");
    for (int i = 0; i < rows; i++) {
        printf("  Fila%2d [", i);
        for (int j = 0; j < cols; j++) {
            printf("%4d", mat[i * cols + j]);
            if (j < cols - 1) printf(", ");
        }
        printf(" ]\n");
    }
}
```

Primero se reserva memoria para el bloque local de cada proceso, que es un bloque de la matriz A con un número de filas igual a `rows_per_proc` y el mismo número de columnas M.

Despues de esto, el proceso 0 distribuye la matriz A entre los procesos utilizando la función `MPI_Scatter()`, la cual se encarga de enviar partes de la matriz a cada proceso, para esto los elementos de esta funcion se componen:

Listing 11: Distribucion de matriz con MPI_Scatter

```c
    local_A = (int *)malloc(rows_per_proc * M * sizeof(int));
    MPI_Scatter(A, rows_per_proc * M, MPI_INT, local_A, rows_per_proc * M,
        MPI_INT, 0, MPI_COMM_WORLD);
```

Donde el primer argumento es el puntero a la matriz A que se va a distribuir, el segundo argumento es el número de elementos que cada proceso recibirá (en este caso, el número de filas por proceso multiplicado por el número de columnas), el tercer argumento especifica el tipo de dato (en este caso, enteros), el cuarto argumento es un puntero al bloque local donde cada proceso almacenará su parte de la matriz, el quinto argumento es nuevamente el número de elementos que cada proceso recibirá, el sexto argumento especifica el tipo de dato para la recepción, el séptimo argumento indica el proceso raíz (en este caso, 0) y el octavo argumento es el comunicador.

Ahora vien sepues de estos e empieza con el calculo de la suma, para esto primero inicializaciamos las variables `fila_global_inicio = id * rows_per_proc;`, la cual se encarga de calcular el índice de la primera fila que cada proceso analizará, tambien tenemos a las variables `local_max_sum` y `local_max_row`, las cuales se encargan de almacenar la suma máxima local y el índice de la fila con la suma máxima local, respectivamente.

Despues de esto se imprime que procesos recibio que pila en especifico, donde la linea de codigo es la siguiente:

Listing 12: Impresion de bloque local recibido por cada proceso

```
        printf("\n  [P%d] Nodo: %s | Recibio filas %d a %d\n",
           id, hostname, fila_global_inicio, fila_global_inicio +
              rows_per_proc - 1);
```

Luego de esto, ahora los procesos lo que haran sera entrar en ciclo for para calcular la suma de cada fila dentro de su bloque local, y determinar la fila con la suma máxima local.

Para esto, el bloque de codigo es el siguiente:

Listing 13: Calculo de suma de filas y determinacion de fila con suma maxima local

```
        for (int i = 0; i < rows_per_proc; i++) {
            int sum = 0;
            int fila_global = fila_global_inicio + i;

            printf("  [P%d] Nodo: %s | Sumando fila %d: ", id, hostname,
               fila_global);
            for (int j = 0; j < M; j++) {
                sum += local_A[i * M + j];
                if (j > 0) printf(" + ");
                printf("%d", local_A[i * M + j]);
            }
            printf(" = %d\n", sum);

            if (i == 0 || sum > local_max_sum) {
                local_max_sum = sum;
                local_max_row = i;
            }
        }
```

En este bloque de codigo, cada proceso recorre las filas de su bloque local, calcula la suma de los elementos de cada fila y determina cuál es la fila con la suma máxima local. Para esto, se utiliza un ciclo for para iterar sobre las filas del bloque local, y dentro de este ciclo, se calcula la suma de los elementos de cada fila utilizando otro ciclo for anidado.

La estructura de funcionamiento del ciclo for anidado, es por medio de un contador de filas locales, el cual se encarga de recorrer las filas del bloque local, y dentro de este ciclo, se tiene otro ciclo for que recorre las columnas de la fila actual para calcular la suma de los elementos de esa fila.

Ademas, se imprime el proceso que esta realizando la suma, el nodo en el que se encuentra, la fila que esta sumando y los elementos de esa fila, junto con el resultado de la suma. Luego de esto, se compara la suma calculada con la suma máxima local almacenada en la variable `local_max_sum`, y si la suma actual es mayor, se actualiza la suma máxima local y el índice de la fila con la suma máxima local.

Siguiendo con las demas linea de codigo, tenemos la inicializacion de la variables `global_max_row = fila_global_inicio + local_max_row;`, donde se calcula el índice global de la fila con la suma máxima local, sumando el índice de la primera fila del bloque local con el índice de la fila con la suma máxima local dentro de ese bloque.

Luego de esto, se imprime el resultado local de cada proceso, indicando cuál es la fila con la suma máxima local y el valor de esa suma.

Listing 14: Impresion de resultado local de cada proceso

```
        int global_max_row = fila_global_inicio + local_max_row;

        printf("  [P%d] Nodo: %s | Mayor suma local -> Fila %d (suma = %d)\n",
            id, hostname, global_max_row, local_max_sum);
        separador();
```

Despues de esto, el proceso 0 se encarga de reservar memoria para almacenar los resultados locales de cada proceso, que consisten en el índice global de la fila con la suma máxima local y el valor de esa suma máxima local. Para esto, el bloque de codigo es el siguiente:

Listing 15: Reserva de memoria para resultados locales en proceso 0

```
        int *local_max_rows = NULL;
        int *local_max_sums = NULL;
        if (id == 0) {
            local_max_rows = (int *)malloc(np * sizeof(int));
            local_max_sums = (int *)malloc(np * sizeof(int));
        }
```

En seguida, se utiliza la función `MPI_Gather()` para recopilar los resultados locales de cada proceso en el proceso 0.

Para esto, el bloque de codigo es el siguiente:

Listing 16: Recopilacion de resultados locales con MPI_Gather

```
        MPI_Gather(&global_max_row, 1, MPI_INT, local_max_rows, 1, MPI_INT, 0,
            MPI_COMM_WORLD);
```

```
        MPI_Gather(&local_max_sum, 1, MPI_INT, local_max_sums, 1, MPI_INT, 0,
            MPI_COMM_WORLD);
```

Para la primera linea de `MPI_Gather()`, tenemos que el primer argumento es un puntero al dato local que cada proceso va a enviar (en este caso, el índice global de la fila con la suma máxima local), el segundo argumento es el número de elementos que cada proceso enviará (en este caso, 1), el tercer argumento especifica el tipo de dato (en este caso, enteros), el cuarto argumento es un puntero al arreglo donde el proceso raíz almacenará los datos recopilados (en este caso, `local_max_rows`), el quinto argumento es nuevamente el número de elementos que cada proceso enviará, el sexto argumento especifica el tipo de dato para la recepción, el séptimo argumento indica el proceso raíz (en este caso, 0) y el octavo argumento es el comunicador. La segunda linea de `MPI_Gather()` es similar, pero en este caso se recopilan los valores de las sumas máximas locales en el arreglo `local_max_sums`.

Por ultimo tenemos el final del programa, donde por medio de una condicional se veririfica que el que imprima el resultado y calcula la ultima parte sea el procesos 0, donde al entrar a la conficional, el proceso 0 imrpimira los resiltados e imprimira un anuncio que el proceo 0 recolecto los resultados y luego de esto, se entra en un ciclo for para determinar cuál es la fila con la suma máxima global, comparando las sumas máximas locales recopiladas y sus correspondientes índices de fila.

Finalmente, el proceso 0 imprime el resultado final, indicando cuál es la fila con la suma máxima global y el valor de esa suma, esto lo hace por medio de la siguiente linea de codigo:

Listing 17: Impresion de resultado final en proceso 0

```
        if (id == 0) {
            printf("\n");
            separador();
            printf("  RESULTADO FINAL\n");
            separador();
            printf("  [P0] Nodo: %s | Recolecto resultados de %d procesos (
                MPI_Gather)\n\n", hostname, np);

            int best_sum = all_sums[0];
            int best_row = all_rows[0];

            for (int i = 0; i < np; i++) {
                printf("  -> P%d reporta: Fila %d con suma %d", i, all_rows[i
                    ], all_sums[i]);
                if (i == 0 || all_sums[i] > best_sum) {
                    if (i > 0) {
                        best_sum = all_sums[i];
```

```
                        best_row = all_rows[i];
                    }
                }
                printf("\n");
            }

            printf("\n  >>> La fila con mayor suma es la FILA %d con suma = %d <<<\n", best_row, best_sum);
            separador();

            free(all_sums);
            free(all_rows);
        }
```

La conficional dentro del ciclo for se encarga de comparar las sumas máximas locales recopiladas y actualizar la suma máxima global y el índice de la fila con la suma máxima global si se encuentra una suma mayor, el orden de impresion es por medio de un ciclo for que recorre los resultados recopilados y los imprime, indicando qué proceso reportó cada fila y su suma correspondiente. Finalmente, se imprime el resultado final indicando cuál es la fila con la suma máxima global y el valor de esa suma, esta se almaceno en las variables `best_row` y `best_sum` respectivamente. Finalmente el proceso 0 libera la memoria asignada para los arreglos de resultados locales.

Resultados:

Al contruir el cluster, primero se verifico que se tuvieran el numero de clientes indicados como minimo que fueron 2, por medio del comando:

```
        cat /etc/hosts
```

Donde la salida fue la siguiente:

```
        user@pel1:~$ cat /etc/hosts
        127.0.0.1   localhost
        10.11.12.1  pel1
        # The following lines are desirable for IPv6 capable hosts
        ::1     localhost ip6-localhost ip6-loopback
        fe00::0 ip6-localnet
        ff00::0 ip6-mcastprefix
        ff02::1 ip6-allnodes
        ff02::2 ip6-allrouters
        ff02::3 ip6-allhosts
        10.11.12.19 pel19
```

```
10.11.12.35 pel35
```

Donde se puede observar que se tienen los clientes pel19 y pel35, por lo que se tiene el numero de clientes necesarios para la realizacion del proyecto.

Despues de esto, se crea el programa por medio de la instruccion nano:

```
user@pel1:~$ nano proyecto_b.c
```

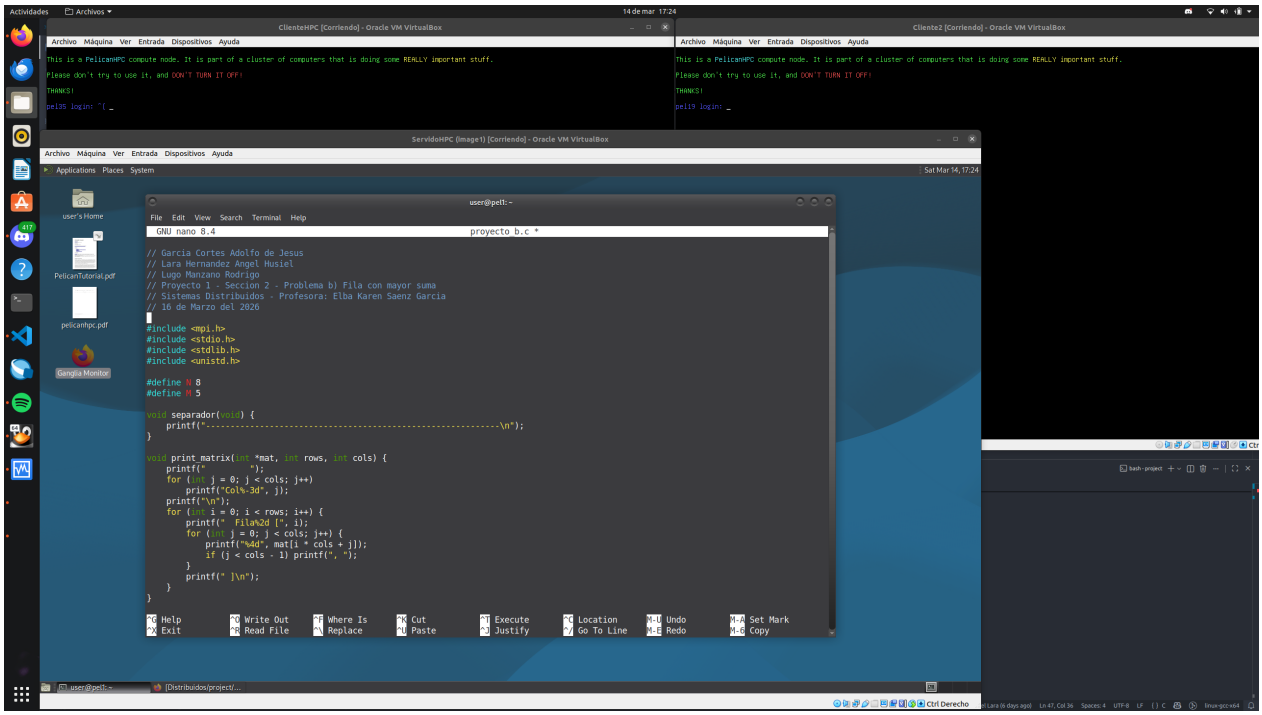

Figura 31: Se crea el programa b) utilizando el editor de texto nano

compilamos el programa es por medio del siguiente comando:

Listing 18: Compilacion de programa b)

```
mpicc proyecto_b.c -o proyecto
```

Y para ejecutar el programa, se utiliza el siguiente comando:

Listing 19: Ejecucion de programa b)

```
mpirun -np 4 ./proyecto
```

Las evidencias son las siguientes:

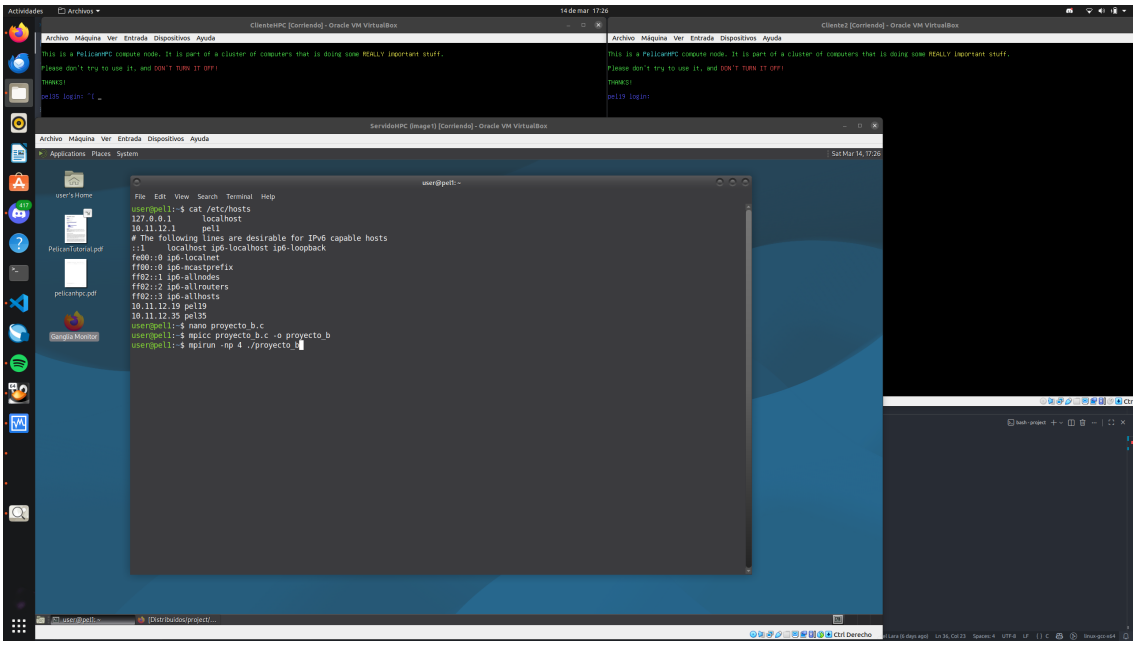

Figura 32: Evidencia de ejecucion del programa b) con 4 procesos

Ahora bien, compilando el programa se puede observar la siguiente salida que estan divida en dos fotos:

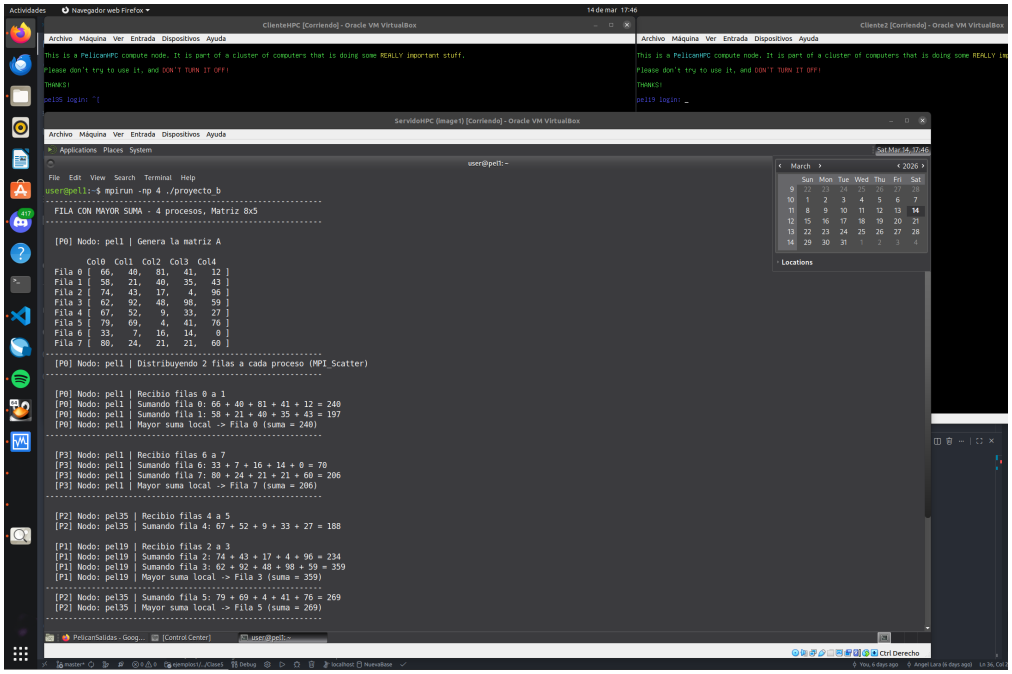

Figura 33: Evidencia de ejecucion del programa b) con 4 procesos (parte 1)

Se ve como la matriz que fue generada por el proceso 0 es la siguiente:

```
        [P0] Nodo: pel1 | Genera la matriz A

                Col0  Col1  Col2  Col3  Col4
        Fila 0 [  66,   40,   81,   41,   12 ]
        Fila 1 [  58,   21,   40,   35,   43 ]
        Fila 2 [  74,   43,   17,   4,   96 ]
        Fila 3 [  62,   92,   48,   98,   59 ]
        Fila 4 [  67,   52, 9,   33,   27 ]
        Fila 5 [  79,   69, 4,   41,   76 ]
        Fila 6 [  33,   7,   16,   14,   0 ]
        Fila 7 [  80,   24,   21,   21,   60 ]
```

Despues de esto, se ve como los clientes reciben cada seccion de la matriz cada uno respectivamente.

```
        [P0] Nodo: pel1 | Recibio filas 0 a 1
        [P1] Nodo: pel19 | Recibio filas 2 a 3
        [P2] Nodo: pel35 | Recibio filas 4 a 5
```

Seguidamente, se tiene la segunda parte de la salida del programa, que termina con la salida del resultado final, donde se puede observar que la mayor suma es de 359:

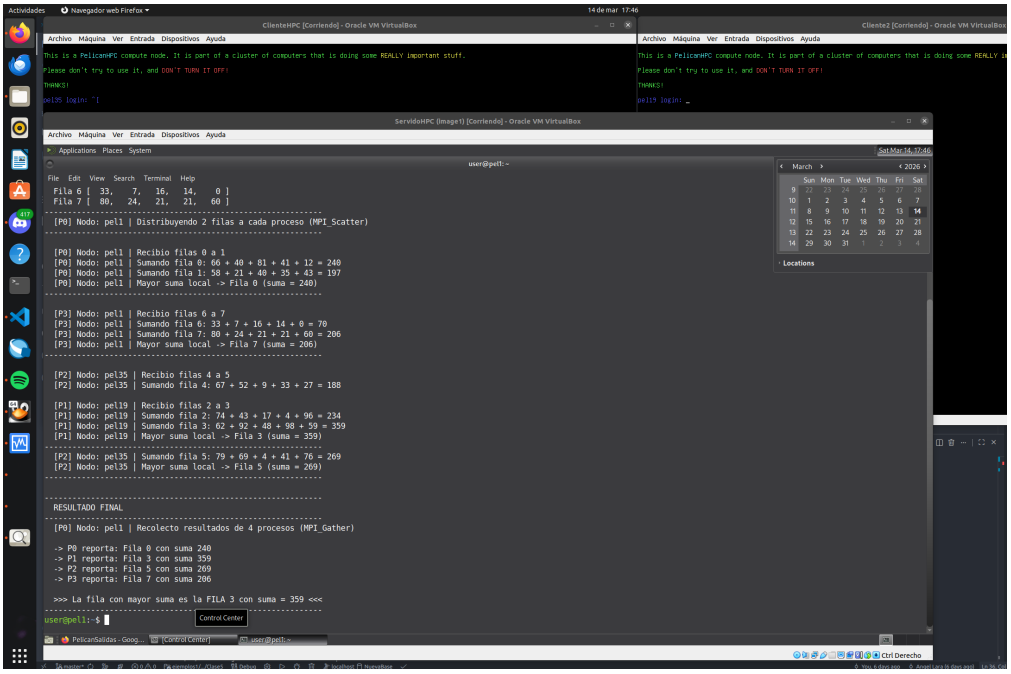

Figura 34: Evidencia de ejecucion del programa b) con 4 procesos (parte 2)

La salida muestra que el proceso 0 genera la matriz de 8×5 y la reparte en bloques de 2 filas por proceso mediante `MPI_Scatter`. Posteriormente, cada proceso calcula la suma de sus filas locales y reporta su mejor resultado: P0 obtiene fila 0 con 240, P1 obtiene fila 3 con 359, P2 obtiene fila 5 con 269 y P3 obtiene fila 7 con 206. Finalmente, el proceso 0 recopila esos resultados con `MPI_Gather`, compara las sumas locales y determina correctamente que la fila con mayor suma global es la **fila 3** con un valor de **359**.

3. Tabla de participacion en el proyecto:

Integrante	**Participacion**
Garcia Cortes Adolfo de Jesus	En el punto 1, con la creacion del cluster, la configuracion de las maquinas virtuales del servidor así como de los 5 clientes usados , la creación del programa para corrobar el correcto funcionamiento del cluster, así como la documentacion del respectivo punto 1.
Lara Hernandez Angel Husiel	En el punto 2 en la seccion b del problema, con la creacion del programa en c sobre la determinacion que renglon de la matriz tiene la mayor suma de sus elementos, ademas de eso en la contruccion de cluster propio para la correcta ejecucion del problema, asi como la documentacion del respectivo punto de la seccion 2 inciso b.
Lugo Manzano Rodrigo	

Tabla 1: Participacion de integrantes en el proyecto

4. Conclusiones:

- **Lugo Manzano Rodrigo:**
- **Garcia Cortes Adolfo de jesus:**
- **Lara Hernandez Angel Husiel:**

Referencias

Anaya, R. (s.f.). *Manual de recursos y aplicaciones Plataforma Rasberry Pi*. https://odin.fi-b.unam.mx/micros/docs/Tutoriales%20Raspberry.pdf

Elahi, A. (2022, 17 de marzo). *Computer systems: Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language* (2.ª ed.). Springer. https://doi.org/10.1007/978-3-030-93449-1

Harris, D., & Harris, S. (2015, 22 de abril). *Digital Design and Computer Architecture* (Arm Edition). Morgan Kaufmann Pub.

Smith, S. (2019, octubre). *Raspberry Pi Assembly Language Programming*. Apress. https://doi.org/10.1007/978-1-4842-5287-1